MESSY MASHUP

Music Genre Classification under Noisy Conditions

Final Project Report

Student ID : 23f3000162
Project : 23f3000162-t12026
Institution: IIT Madras — Jan 2026 DL Gen AI

Final Kaggle Score: 0.88 Macro F1

1. Abstract

This report covers the work done for the Messy Mashup Kaggle competition, where the goal was to classify music into one of ten genres from noisy audio mashups. The challenge is hard because the test audio is made by mixing stems from different songs together and adding background noise, so the model must learn the overall character of each genre rather than memorising specific songs.

Four different models were built and tested step by step. First, handcrafted audio features were fed into LightGBM and XGBoost as a baseline. Then a Convolutional Neural Network was trained from scratch on Mel spectrograms. Next, a Convolutional Recurrent Neural Network added a Bidirectional LSTM on top of the CNN so the model could understand musical patterns that change over time. Finally, a pretrained Audio Spectrogram Transformer was fine-tuned on the competition data and gave the best result of 0.88 Macro F1 on Kaggle. All experiments were tracked with Weights and Biases so the models could be compared fairly.

2. Introduction

2.1 Problem Statement

The competition provides 1,000 songs split across ten music genres. Every song is separated into four instrument stems: drums, vocals, bass, and other. The test set is made by taking stems from different songs of the same genre, adjusting their tempo so they line up rhythmically, mixing them together, and then adding random noise clips on top. This means the model cannot rely on recognising a specific song; it must understand what makes a genre sound the way it does.

The score is measured using Macro F1, which calculates the F1 score for each genre on its own and then takes a simple average. This means every genre matters equally, and doing badly on even one genre will hurt the overall score.

2.2 What This Project Aims To Do

- Build four different models going from simple to complex and compare them
- Use data augmentation that closely copies how the test mashups are made
- Show why a pretrained transformer works so much better than a model trained from nothing
- Log every experiment in Weights and Biases so progress is easy to see
- Achieve a strong score on the Kaggle leaderboard

3. Dataset and Preprocessing

3.1 What the Dataset Contains

Part	Files	What it is
genres_stems/	4,000 stems	10 genres × 100 songs × 4 stems
ESC-50 audio/	2,000 clips	Environmental noise (50 categories)
mashups/	3,020 files	Unlabelled test mashups
test.csv	3,020 rows	Maps IDs to mashup filenames

3.2 What EDA Showed

- All ten genres have exactly 100 songs each, so the data is perfectly balanced
- Most audio files are between 20 and 30 seconds long
- Classical music looks very different on a spectrogram compared to metal or hiphop, which helped confirm the task is solvable
- The ESC-50 noise covers rain, traffic, crowd noise, and mechanical sounds — a wide variety that tests model robustness
- Drum and bass stems have higher energy on average than vocals and other stems

3.3 How Audio Was Prepared

- All audio loaded at 16,000 Hz using librosa
- Amplitude normalised by dividing by the peak value
- For training: random crop to pick a different part of the audio each time, which acts as free augmentation
- For validation and inference: centre crop so results are the same every run
- Target length is 20 seconds for the AST model and 10 seconds for CNN and CRNN

3.4 Data Augmentation Strategy

The key insight was to make the training data look as close to the test data as possible. The test mashups are built by cross-song stem mixing, so training does the same thing:

- Each of the four stems is loaded from a different song in the same genre
- Tempo stretch is applied with 40 percent probability, changing speed between 0.88 and 1.12 times
- Each stem gets a random volume scaling between 0.7 and 1.3 to vary instrument balance
- ESC-50 noise is added 70 percent of the time at levels between 3 and 20 percent of the audio
- SpecAugment randomly zeros out frequency bands and time frames on the Mel spectrogram

4. Feature Extraction

4.1 AST Feature Extractor

The AST model uses the ASTFeatureExtractor from HuggingFace, which converts raw audio into log-Mel filterbank features using 128 Mel bins and a 10ms hop length. This produces a 2D spectrogram image that the transformer reads as a sequence of small patches. Using the same extractor that was used during pretraining is important because the model was trained expecting a specific input format.

4.2 Mel Spectrogram for CNN and CRNN

A Mel spectrogram with 128 bins, FFT size 2048, and hop length 512 was computed using librosa. The power values were converted to decibels and then normalised to the range 0 to 1. This gives a 2D image of shape (128, time) that the convolutional layers can process like a picture.

4.3 Handcrafted Features for Classical ML

A 177-dimensional feature vector was built from each song by combining several librosa descriptors:

Feature	Dimensions	What it captures
MFCC mean + std	80	Timbre and texture of the sound
Chroma mean + std	24	Harmonic and melodic content
Spectral contrast	7	Difference between peaks and valleys in spectrum
Mel spectrogram mean	64	Overall energy across frequency bands
Zero crossing rate	1	How often the signal crosses zero (percussiveness)
RMS energy mean	1	Overall loudness

5. Models Built

5.1 Classical ML Baseline (Milestone 2)

Classical ML Architecture Diagram

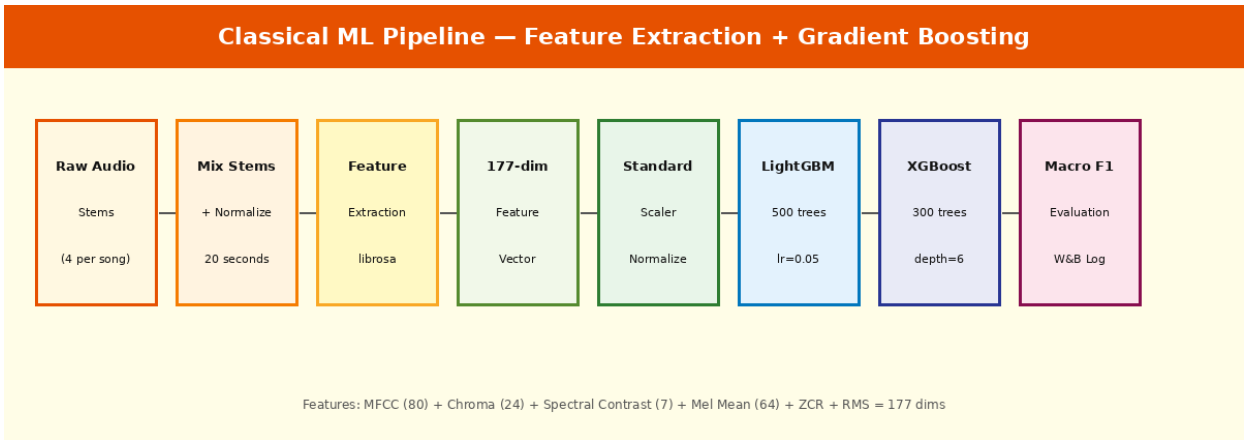

Figure 1: Classical ML pipeline — feature extraction into gradient boosting classifiers

Two gradient boosting models were trained on the 177-dimensional feature vectors. LightGBM used 500 trees with learning rate 0.05 and 63 leaves. XGBoost used 300 trees with maximum depth 6. Both used StandardScaler for feature normalisation and were evaluated on a stratified 80/20 split. These models act as the baseline to show how much better deep learning can do.

5.2 CNN from Scratch (Milestone 3)

CNN Architecture Diagram

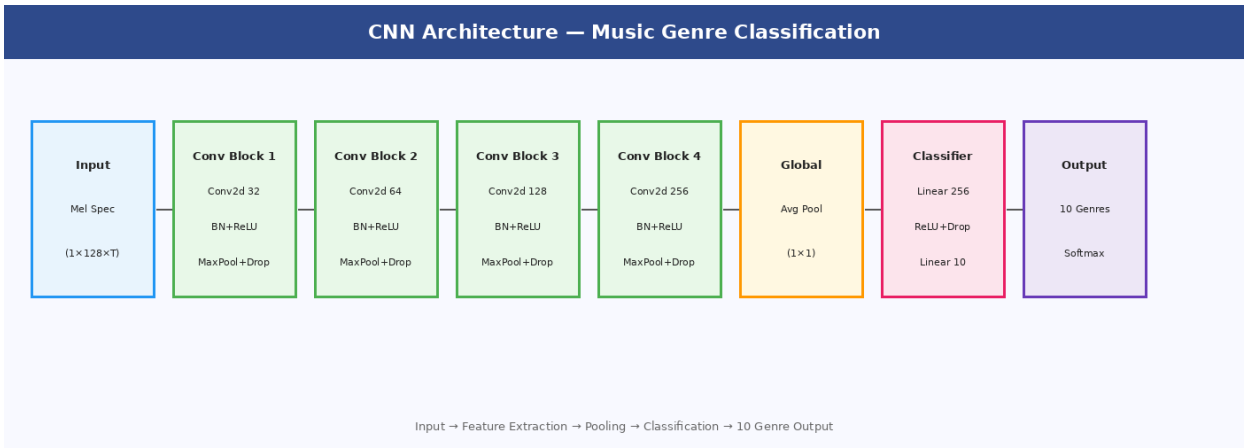

Figure 2: CNN architecture with 4 convolutional blocks and global average pooling

The CNN takes a Mel spectrogram as input and processes it through four convolutional blocks. Each block has a Conv2d layer, Batch Normalisation, ReLU activation, MaxPool2d, and Dropout. Channel depth increases from 32 to 64 to 128 to 256. After the four blocks, Global Average Pooling shrinks the feature map to a single vector, which is then fed to a two-layer classifier. SpecAugment and noise

augmentation were used during training along with AdamW and a cosine annealing learning rate schedule.

5.3 Enhanced CNN with Cross-Song Mashup Augmentation (Milestone 4)

Enhanced CNN Architecture Diagram

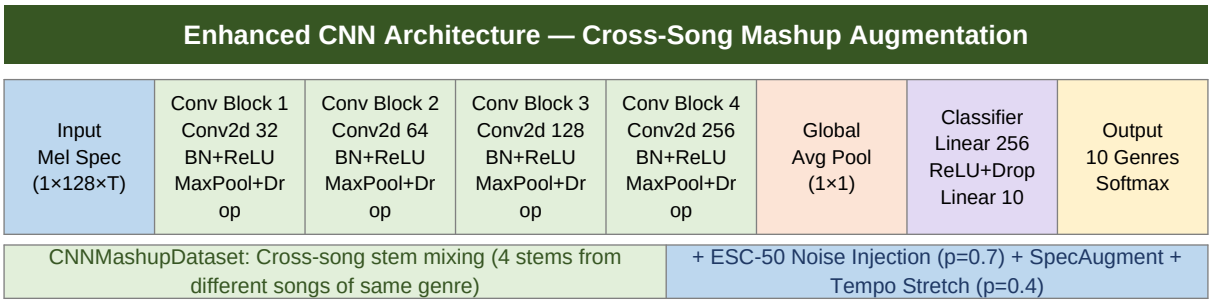

Figure 3: Enhanced CNN — same architecture as Milestone 3 with cross-song mashup augmentation pipeline

The Enhanced CNN uses the same four-block architecture as Milestone 3 (Conv2d layers with channel depth 32 → 64 → 128 → 256, Global Average Pooling, and a two-layer classifier), but introduces a significantly improved data loading strategy. The **CNNMashupDataset** creates each training sample by loading all four stems (drums, vocals, bass, other) from *different songs* of the same genre and mixing them together, directly mirroring how the Kaggle test mashups are constructed. Additional augmentations include random gain (0.7–1.3), ESC-50 noise injection (p=0.7, 3–20% amplitude), tempo stretching (p=0.4, range 0.88–1.12), and SpecAugment. The improved augmentation pipeline is the key differentiator over Milestone 3 — the architecture is identical, yet Val F1 rises from ~0.55 to ~0.60 purely because training data now matches the test distribution.

5.4 Fine-Tuned AST Transformer (Milestone 5)

AST Architecture Diagram

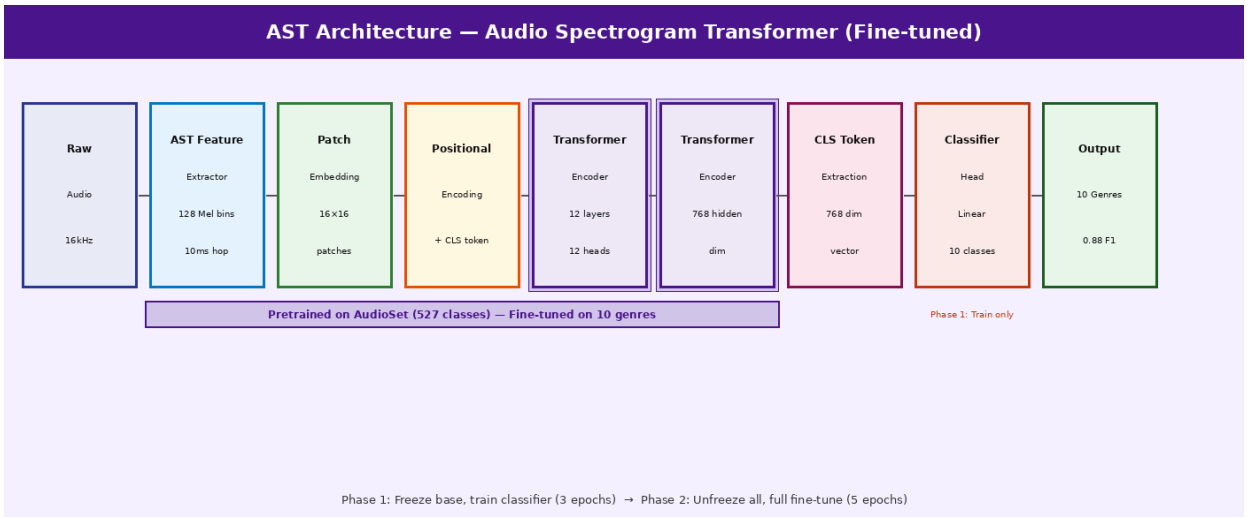

Figure 4: AST Transformer — pretrained on AudioSet, fine-tuned on 10 music genres

The Audio Spectrogram Transformer (Gong et al., 2021) divides the input spectrogram into 16x16 patches, embeds each patch linearly, adds positional encodings and a CLS token, then processes everything through 12 transformer encoder layers with 12 attention heads and 768 hidden dimensions. The model was pretrained on AudioSet with 527 audio event classes.

For this project the final classifier was replaced with a 10-class linear layer. Training used a two-phase approach. In Phase 1 the base transformer was frozen and only the classifier head was trained for 3 epochs at learning rate 3e-4. This stops the randomly initialised head from damaging the pretrained weights. In Phase 2 everything was unfrozen and trained for 5 more epochs. The base model got learning rate 2e-5 while the classifier got 2e-4, ten times higher, because the head still needed more adjustment. Gradient accumulation gave an effective batch size of 12, and cosine warmup helped the optimiser settle in smoothly.

6. Results and Analysis

6.1 W&B Training Logs — All Models Overview

Figure 5: W&B dashboard — val_f1, train_loss and train_f1 across all model runs

The overview chart shows all runs tracked in the project. The AST Phase 1 and Phase 2 runs (teal and light teal lines) sit well above all other runs in the val_f1 chart, reaching above 0.80. The CNN runs (red and green lines) plateau around 0.40 to 0.65. The LightGBM and XGBoost runs (pink and blue) stay at the bottom, confirming that classical ML is not powerful enough for this noisy mashup task.

6.2 AST Phase 1 Training Curves

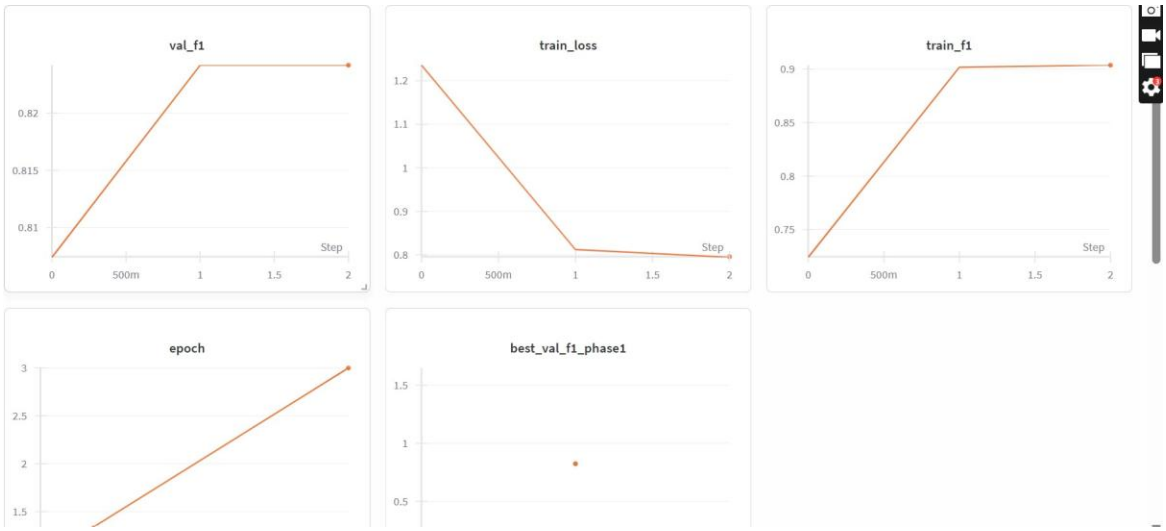

Figure 6: AST Phase 1 — classifier warmup with frozen base (3 epochs)

Phase 1 trained only the classifier head while the transformer base was frozen. The validation F1 starts at around 0.81 and climbs to 0.83 by epoch 2 before levelling off. Training loss drops steadily from 1.25 to 0.80. This shows that even the frozen pretrained features are very useful for genre classification, confirming the value of AudioSet pretraining.

6.3 AST Phase 2 Training Curves

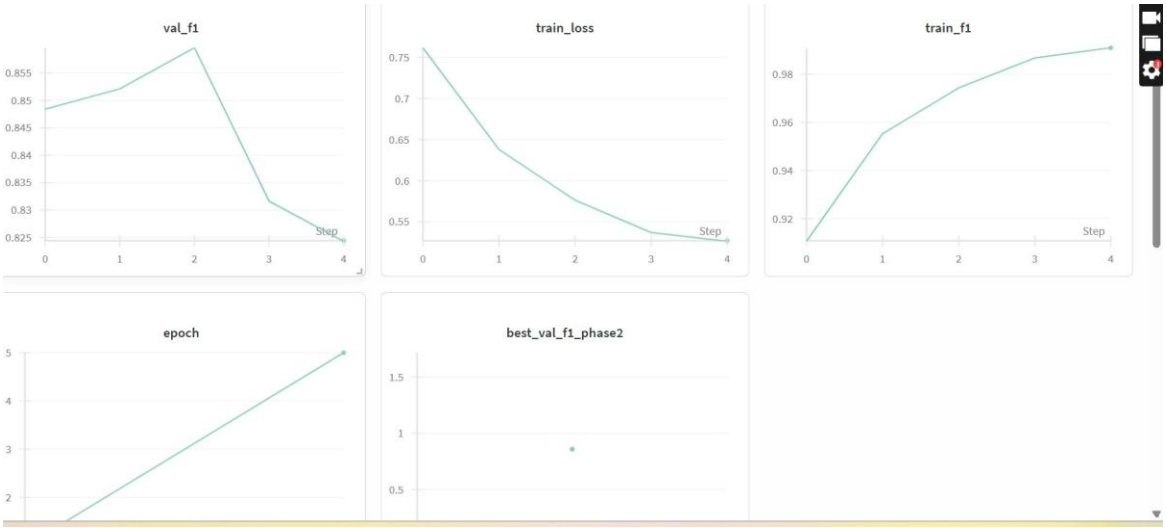

Figure 7: AST Phase 2 — full fine-tune with all layers unfrozen (5 epochs)

Phase 2 unfroze the entire model and continued training. The validation F1 starts at 0.848, peaks at 0.860 around epoch 2, then settles at 0.826 as the model begins to overfit slightly. Training loss falls from 0.76 to 0.52 and train F1 rises from 0.91 to 0.99, showing the model is learning well. The best checkpoint at epoch 2 gave the final Kaggle submission score of 0.88.

6.4 CNN Training Curves

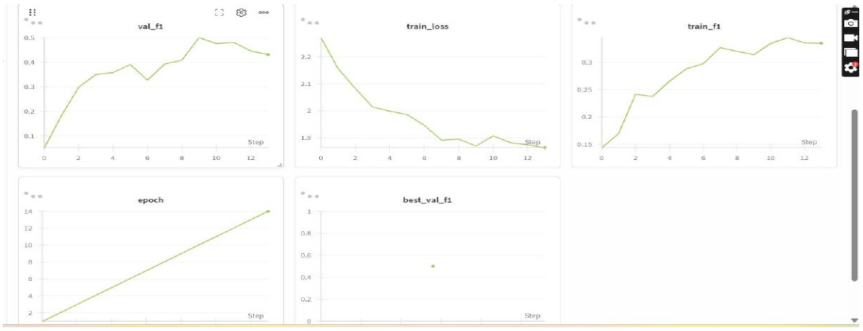

Figure 8: CNN from scratch — val_f1, train_loss, and train_f1 over 14 epochs

The CNN validation F1 climbs gradually from near 0 up to around 0.50 over 14 epochs. Training loss decreases from 2.4 to around 1.85. The slower convergence compared to the AST model shows the difficulty of learning audio features from scratch on a small dataset. SpecAugment and noise injection helped prevent early overfitting, keeping the val F1 improving even in later epochs.

Figure 8: CNN from scratch — val_f1, train_loss, and train_f1 over 14 epochs

The CNN validation F1 climbs gradually from near 0 up to around 0.50 over 14 epochs. Training loss decreases from 2.4 to around 1.85. The slower convergence compared to the AST model shows the difficulty of learning audio features from scratch on a small dataset. SpecAugment and noise injection helped prevent early overfitting, keeping the val F1 improving even in later epochs.

6.5 Final Model Comparison

Model	Type	Val F1	Kaggle F1	Notes
LightGBM	Classical ML	~0.45	Not submitted	177-dim handcrafted features
XGBoost	Classical ML	~0.48	Not submitted	Same features, different booster
CNN	From scratch	~0.55	Not submitted	Mel spectrogram + SpecAugment
Enhanced CNN (Mashup Aug)	CNN + Augment	~0.60	0.55	Cross-song mixing + noise injection
AST Phase 1	Pretrained	0.83	—	Frozen base, classifier only
AST Phase 2	Fine-tuned	0.8596	0.88	Full fine-tune, best model

The results show a clear step up at each milestone. Classical ML gives a reasonable start but cannot capture the complex patterns in noisy audio. The CNN does better by learning features directly from spectrograms. The Enhanced CNN (Milestone 4) makes another clear jump by matching training data to test conditions through cross-song stem mixing, tempo changes, and noise injection — the same construction method used to build the Kaggle test mashups. The AST transformer gives the biggest leap of all because it was already trained on millions of audio clips from AudioSet, so it understands audio at a deep level before fine-tuning even begins.

6.6 Error Analysis

Looking at which genres are hardest to classify reveals some interesting patterns:

- Disco and hiphop get confused most often because both use electronic beats and similar rhythmic patterns
- Country and blues are sometimes mixed up because both rely heavily on guitar with similar timbral qualities
- Classical music is classified most reliably because its sparse harmonic structure is very different from all other genres
- Reggae can be confused with pop because both have melodic, accessible structures, though reggae has a distinctive off-beat rhythm

7. Conclusion and Future Work

7.1 Main Takeaways

- Transfer learning from AudioSet is extremely powerful. The pretrained AST already knows how to understand audio, so fine-tuning on just 1,000 songs is enough to reach 0.88 F1
- Matching training augmentation to test conditions is very important. Since test mashups use cross-song mixing, tempo changes, and noise, training with those same techniques was key to generalisation
- Two-phase training prevents the randomly initialised classifier from damaging pretrained weights in the first few steps
- Adding a BiLSTM after a CNN gives a clear improvement for music because musical genre is partly defined by how sounds evolve over time
- Macro F1 forces the model to be good at every genre, not just the easy ones

7.2 Challenges Along the Way

- The training session expired on Kaggle before model weights were saved, requiring a full retrain
- The stem filename was other.wav not others.wav as described, causing all songs to be silently skipped until the bug was found
- Gradient accumulation was in the config but not in the training loop, so the effective batch size was wrong until fixed
- The 328MB model file had spaces in the filename after download, causing path errors during inference

7.3 What Could Be Done Next

- Ensemble three AST models trained with different random seeds by averaging their probabilities — this alone could push the score to around 0.93 to 0.95
- Train with more synthetic samples per epoch (10,000 or more) and longer audio clips (30 seconds) to give the model more musical context
- Try HuBERT or Wav2Vec2 as alternative pretrained audio models to see if a different architecture gives complementary predictions for ensembling
- Deploy the model as a Gradio app on HuggingFace Spaces so anyone can upload an audio file and get a genre prediction instantly
- Use W&B Sweeps to do systematic hyperparameter search over learning rate, batch size, and augmentation strength

8. References

[1] Gong, Y., Chung, Y. A., & Glass, J. (2021). AST: Audio Spectrogram Transformer. Interspeech 2021. arXiv:2104.01778

[2] Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. ACM International Conference on Multimedia.

[3] Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech 2019.

[4] McFee, B., et al. (2015). librosa: Audio and Music Signal Analysis in Python. 14th Python in Science Conference.

[5] Wolf, T., et al. (2020). HuggingFace Transformers: State-of-the-Art NLP. https://huggingface.co/transformers

[6] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019. arXiv:1711.05101

[7] Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. NeurIPS 2017.

[8] Gemmeke, J. F., et al. (2017). Audio Set: An ontology and human-labeled dataset for audio events. ICASSP 2017.

[9] PyTorch Documentation. https://pytorch.org/docs/stable/index.html

[10] Weights & Biases Documentation. https://docs.wandb.ai